해태

Haetae

옳고 그름을 가리는 RPKI 검증기

해태는 옳고 그름을 가린다는 상상의 동물입니다.
이 해태는 라우터가 받는 경로가 옳은지를 가립니다.



2026-11-11 · 0.3.0 · 시험 운영 중 · 오픈 소스 (ISC)

01 한눈에

라우터가 믿을 경로 정보를, 독립된 두 번째 눈으로 가립니다

0
rpki-client 와의 차이
같은 입력에서 VRP · ASPA · 라우터 키

약 40초
전체 검증
객체 약 57 만 개 (rpki-client 16 분)

0
외부 라이브러리
Go 표준 라이브러리만

1
실행 파일
수집 · 검증 · RTR · API 를 하나로

검증 모든 표준 RPKI 객체 (ROA · ASPA · BGPsec 키 등 9 종)

제공 라우터에는 RTR v0 / v1 / v2, 시스템에는 HTTP API

운영 10 분 주기 상주 · 매일 Routinator · rpki-client 와 대조

RPKI : 이 경로를 이 AS 가 내도 되는가

자원 보유자

RIR 에서 받은 주소에 ROA ("이 prefix 는 이 AS 가 냄") 를 서명해 저장소에 게시

RP (Haetae)

전 세계 저장소에서 받아 서명 · 유효기간 · 자원 범위를 확인하고 VRP 목록을 만듦

라우터

BGP 로 받은 경로를 VRP 와 대조해 판정, 정책에 따라 받거나 버림

Valid

VRP 와 AS · 길이가 맞음
받음

Invalid

덮는 VRP 는 있으나 AS 나 길이가 다름
버림 (하이재킹 · 오설정 차단)

NotFound

덮는 VRP 가 없음
지금처럼 받음

VRP = (AS, prefix, 최대 길이). ASPA 는 AS 사이의 고객 · 공급자 관계로 경로 누출을 막고, BGPsec 키는 경로 서명을 확인합니다

03 왜

RP 하나의 실수는 곧 라우팅 장애입니다

라우터는 RP 가 준 목록대로 경로를 버립니다

- RP 의 버그나 장애 = 정상 경로가 Invalid 로 바뀌어 사라짐
- 서로 다른 구현의 RP 를 둘 두면 한쪽의 문제를 다른 쪽이 드러냄
- 그런데 운영에 쓸 만한 공개 RP 는 많지 않음 (오른쪽)

Haetae 는 기존 RP 와 코드를 나누지 않는 새 구현입니다

Routinator	Rust · NLnet Labs	운영 중
rpki-client	C · OpenBSD	운영 중
FORT	C · NIC.mx	운영 중
rpki-prover	Haskell · 개인 개발	운영 중
OctoRPKI	Go · Cloudflare	보관됨
RIPE NCC Validator	Java · RIPE NCC	종료

04 구조

받고, 가리고, 내어 줍니다 : 10 분마다 한 회차

1 수집

- RRDP 우선, 오래 실패하면 rsync
- 저장소 동시 32 곳
- 저장소당 5 분, 회차 20 분 기한

2 검증

- TAL → TA → CA → Manifest
- 서명 · 해시 · CRL · 유효기간
- 자식 자원은 부모 안에 (RFC 3779)

3 집합

- VRP · ASPA · 라우터 키
- SLURM 로컬 예외
- serial 과 변경분 (delta)

4 제공

- RTR v0 / v1 / v2
- HTTP API (조회 · 변경분)
- JSON · CSV 파일

검증에 실패해도 직전 정상 결과를 계속 주고, 판을 올려도 라우터 연결이 끊기지 않습니다

RFC 6487 · 3779 · 6488 · 9286 · 9582 · 8182 · 8210 · 8416 · 8630 · 9691

저장소에 게시되는 표준 객체를 하나도 빼지 않고 검증합니다

약 57만

검증한 객체 (2026-11)

모든 객체의 종류 · URI · 해시 · 유효기간 · 소속 CA · 상태 · 내용 요약을 JSON 으로 조회

CA 인증서 RFC 6487 · 3779 56,164	**CRL** RFC 6487 56,078	**Manifest** RFC 9286 56,078
ROA → VRP RFC 9582 400,187	**ASPA** draft aspa-profile 4,059	**BGPsec 라우터 키** RFC 8209 3
GBR 연락처 RFC 6493 1	**SPL** draft rpki-prefixlist 0 (게시 전)	**TAK** RFC 9691 0 (게시 전)

표준이 아닌 확장자 (사설 · 시험 객체) 는 개수만 셈 · RSC (RFC 9323) 는 파일 하나를 확인하는 모드로

06 정확성

같은 입력이면 rpki-client 와 차이 0 입니다

0

같은 캐시 · 같은 검증 시각

- VRP 1,029,840 · ASPA 3,904 · 라우터 키 모두 같음 (고정 스냅샷, 객체 약 57 만)
- Haetae 가 직접 받은 캐시를 rpki-client 로 검증해도 같음
- 코드를 바꿀 때마다 이 스냅샷으로 다시 확인

매일 비교 (2026-11-10) : 각자 따로 수집

Haetae	1,033,062	
Routinator	1,033,060	Haetae 와 2 개 차이
rpki-client	1,033,047	Routinator 와 17 개 차이

남은 차이는 수집 시각 차이 (몇 분 ~ 수십 분) 사이에 바뀐 ROA 입니다. 다른 두 RP 끼리의 차이보다 작습니다

07 속도

16 분 걸리던 검증을 약 40 초에



같은 서버 · 같은 캐시 (객체 약 57 만) 오프라인 검증, 각각 혼자 실행 · rpki-client 9.0 = Ubuntu 24.04 패키지 · Haetae 는 검증을 여러 코어에 나눔

약 1 분
수집 + 검증 한 회차 (10 분 주기)

약 2 초
판 올리기 때 RTR 멈춤 (연결은 그대로)

약 5 GB
상주 메모리 (회차 중 최대)

08 기준

TA constraints 를 쓰지 않습니다 : RFC 그대로

Ubuntu 24.04 패키지 파일 (2024-04)

2,773 VRP · 59 ASPA

정상 객체를 거부

최신 파일 (2026-07) 로 바꾼 뒤

차이 0

Haetae 와 같은 결과

rpki-client 는 RIR 별 자원 범위 파일 (.constraints) 로 객체를 거릅니다

- RFC 에 없는 rpki-client 고유 기능, 파일은 rpki-client 개발자가 만들어 함께 배포
- 파일이 IANA 할당보다 늦으면 새로 할당된 자원의 정상 객체가 거부됨 (위 : 2024-11 APNIC 할당 2410::/12 가 없음)
- Haetae 는 쓰지 않음 (Routinator · FORT 와 같음) · RIR 의 TA 가 모든 자원을 가지는 것이 표준 운영

2026-10 측정 · 같은 데이터, 같은 rpki-client 9.0 에서 constraints 파일만 바꿔 검증 · 차이 = constraints 없이 검증한 결과와 비교

라우터에는 RTR v0 · v1 · v2 로 줍니다

v0
RFC 6810
가장 오래된 장비까지 (VRP)

v1
RFC 8210
BGPsec 라우터 키 더함

v2
draft 8210bis
ASPA 더함 (경로 누출 방지)

변경분 Serial Notify 로 바뀐 것만 · 재시작 · 판 올리기 뒤에도 session 과 serial 이 이어져 라우터가 전체를 다시 받지 않음

버전 협상 라우터가 붙는 판에 맞춰 v2 → v1 → v0 으로 낮춰 답함

접근 · 보안 허용 대역 · 주소별 연결 상한 · 첫 질의 / 유휴 시간 제한 + 선택 : TCP-MD5 · TCP-AO · TLS · SSH

TCP 3323 (사실상 표준 포트) · Cisco IOS XR 시험 라우터와 연동 · rtrdump 와 자체 도구로 RTR 전체 = JSON 확인

시스템에는 HTTP API 로 줍니다

Routinator 와 같은 모양

- /api/v1/validity (경로 판정)
- /json-delta (바뀐 것만)
- /jsonext (VRP 의 출처 ROA, 선택)

rpki-client 와 같은 모양

- /json (VRP · ASPA · 라우터 키)
- /csv
- 같은 모양이라 비교 도구가 그대로 읽음

Haetae 고유

- AS · prefix 조회
- ASPA 경로 검증 (upstream / downstream)
- 객체 · CA 탐색, 만료 임박
- unsafe VRP, 회차 이력
- /healthz · Prometheus 지표

출발지로 두 단계 : 관리망 (내부) 은 모든 경로, 인터넷 (공개) 은 조회 경로만 · 공개 쪽은 주소마다 연결 · 횟수 제한, 라우터 주소 같은 내부 정보는 뺌

11 안전

입력은 믿지 않고, 실패해도 흔들리지 않게

입력을 믿지 않음

- 크기 · 개수 · 깊이 · 시간 상한
- 거대한 RRDP 요소는 메모리 전에 차단
- https 만, 경로 탈출 차단
- 객체 하나의 오류는 그 객체만

수집이 실패해도

- manifest 가 틀리면 직전 정상본 (RFC 9286)
- 망 오류면 직전 데이터, 죽은 저장소는 간격을 늘림
- 변경분은 모두 확인한 뒤 한 번에
- 회차 기한 : 늦은 저장소는 직전 데이터로

결과가 이상하면

- TA 하나라도 실패하면 반영 안 함
- VRP 가 크게 줄면 경고 (보류 선택)
- 거부된 CA 와 겹치는 VRP 를 unsafe 로 표시
- 1 시간 넘게 갱신이 없으면 경보

root 가 아닌 전용 계정 · systemd 격리 · 파서 6 개 fuzz 각 3 시간 (약 2 억 번, 문제 0) · 전체 코드 검토 두 번 (2026-11)

12 운영

실행 파일 하나, 설정 파일 하나, 멈추지 않는 운영

설정 파일 하나

모든 옵션이 설정 키 · 명령줄이 우선 · 모르는 키나 틀린 값이면 시작 안 함

멈추지 않고 바꾸기

systemctl reload · 새 값을 모두 확인한 뒤 한 번에, 틀리면 지금 값 그대로

TAL 내장

5 RIR 공식 TAL 을 실행 파일 안에 · 다른 패키지에 기대지 않음

무중단 판 올리기

같은 프로세스가 새 판으로 · 라우터 연결 그대로, 약 2 초 멈춤

감시

/healthz (정상 200, 아니면 503) · Prometheus · systemd watchdog

접근 제어

RTR · HTTP 각각 허용 목록 · HTTP 는 내부 / 공개 두 단계

linux amd64 · arm64 실행 파일을 내려받아 바로 (의존성 없음) · systemd 서비스 · 설명서 · API 문서 함께 배포

0.2.0 → 0.3.0 에서 바뀐 것

보안

- DER 개수 · RRDP 요소 크기 상한
- 공개 HTTP : 주소마다 연결 상한, 쓰기 시간 제한
- TLS 인사 실패 때 연결 자리 돌려줌
- systemd 보호 설정 강화
- 내 라우터 대역은 연결 상한을 따로 (rtr.trusted)

안정

- TA 당 저장소 수 상한, 안 쓰는 폴더 정리, 디스크 확인
- 회차 기한 (20 분)
- publication point 메모리 상한
- 객체 하나의 오류는 그 객체만

운영

- TAK 다음 키 예고 경고
- 느린 저장소를 로그에
- 변경분 · RTR 전체를 늘 같은 순서로
- SSH 전송 : 응답을 끝까지 넘김
- linux amd64 · arm64 실행 파일 배포

2026-11 전체 코드 검토 (높음 5 · 중간 10 건) 모두 반영 · 파서 fuzz 추가 실행 · 같은 입력이면 rpki-client 와 차이 0 그대로

14 다음

다음 단계 : 병행 검증을 거쳐 운영으로

끝남 · 공개 · 지금 · 다음

기능

- 모든 표준 객체
- RTR v0 / v1 / v2, 전송 보안
- ASPA 경로 검증 · RSC
- 설정 · reload · TAL 내장

v0.1 → v0.3.0

- GitHub, ISC 라이선스
- 코드 검토 반영
- amd64 · arm64 실행 파일

시험 운영

- 10 분 주기 상주
- 매일 결과 대조
- 시험 라우터 · 인터넷 시험 접속

병행 검증

- 기존 RP 와 몇 주 함께
- 라우터별 RTR 보안 · ASPA 지원 확인
- 운영 적용 판단

해태

Haetae

독립 구현 · 같은 결과 · 흔들리지 않는 운영

Go 표준 라이브러리만으로 만든 RPKI Relying Party · ISC 라이선스 오픈 소스

https://github.com/yunsog/haetae